TERRAFORM · MY LEARNING NOTES · SESSION 01

Terraform Workflow

Your notes, cleaned up from dictation + screenshots — Lecture 1

Quick recall

Terraform Workflow is the general process/best-practice sequence for managing infrastructure with **Terraform** (HashiCorp's Infrastructure as Code tool): define, provision, and manage cloud infrastructure in a **predictable, repeatable** way. It's exactly **5 steps**:

Write → Initialize → Plan → Apply → Destroy

The 5 steps

1. Write — define your infrastructure

You write `.tf` files (Terraform configuration files) containing code that describes the infrastructure you want. The filename can be anything, but the extension **must** be `.tf` — and a meaningful name (not just `main.tf` for everything) makes it obvious what each file is for later.

Example intent: "I want an EC2 server in AWS."

2. Initialize — prepare Terraform

```
terraform init
```

Sets up Terraform inside your project directory and **downloads the plugins your code actually needs** — these are called **providers** (AWS, Azure, GCP, Kubernetes, etc.). Terraform reads what you wrote in step 1 and pulls down only the provider(s) referenced there — write AWS resources, it downloads the AWS provider; write Azure resources, it downloads the Azure provider instead.

3. Plan — preview what will happen

```
terraform plan
```

A **dry run**. Terraform shows exactly what it *plans* to do — what will be **created, changed, or destroyed** — without touching real infrastructure. This is how you catch mistakes **before** anything actually

changes.

4. Apply — make it real

```
terraform apply
```

Terraform actually **creates or updates** your infrastructure so it matches the code you wrote. This is the step where real cloud resources get provisioned.

5. Destroy — release resources

```
terraform destroy
```

Deletes/decommissions **everything Terraform created** for this configuration. Use this when you want to fully tear down what you provisioned.

```
Write (.tf code) → Initialize (terraform init, downloads providers)
                 → Plan (terraform plan, dry run)
                 → Apply (terraform apply, provisions real infra)
                 → Destroy (terraform destroy, tears it all down)
```

Terraform CLI cheat sheet (from this session)

Command	What it does
`terraform version`	Display the Terraform binary version
`terraform --help`	Help page (add `-help` after a subcommand for less common/complex flags)
`terraform init`	Initialize the working directory, pull down providers
`terraform validate`	Validate the code for syntax correctness
`terraform fmt`	Reformat code to canonical HCL style
`terraform providers`	Show information about the providers required by this config
`terraform plan`	Create an execution plan (dry run)
`terraform apply`	Execute changes to reach the desired state
`terraform apply --auto-approve`	Apply changes without the interactive "yes" confirmation prompt
`terraform show`	Provide human-readable output of the current state

`terraform state list`	List every resource currently tracked in state
`terraform state show <resource>`	Show the full attributes of one specific resource in state
`terraform destroy`	Destroy everything Terraform is tracking in this configuration
`terraform destroy --target <resource>`	Destroy **only** the named resource, leaving the rest intact

---

Practical walkthrough from this session

Worked with a config that provisioned an EC2 instance (`ec2.tf`) and later an S3 bucket, running through the real command history:

```
terraform init
terraform plan

# added Terraform's shell autocompletion, then reloaded the shell
vim .bashrc
source ~/.bashrc

terraform plan
terraform version
terraform --help
terraform validate
terraform fmt               # reformats ec2.tf to canonical style if it drifted
ls
cat ec2.tf                  # sanity-check the actual resource code before applying
terraform providers

terraform apply --auto-approve

terraform state list        # see every resource Terraform now tracks
terraform show              # full human-readable current state
terraform state show aws_instance.web   # attributes of just this one resource
```

Destroying selectively, one resource at a time:

```
terraform destroy --target aws_instance.web
terraform state list                          # confirm only that resource is gone
terraform destroy --target aws_s3_bucket.my_s3_bucket
```

`--target` is the way to tear down **one specific resource** without destroying the entire stack — useful when you provisioned multiple independent things (an EC2 instance *and* an S3 bucket here) and only want to remove one of them.

Practice Tasks

☐ Write a minimal `.tf` file for a single resource (e.g. an S3 bucket), run `terraform init`, and confirm the correct provider plugin gets downloaded automatically based on the resource type you wrote.

☐ Run `terraform plan` before ever applying, and get in the habit of reading it fully — confirm you can predict exactly what `apply` will do from the plan output alone.

☐ Apply a config with two independent resources, then use `terraform state list` + `terraform state show <resource>` to inspect each one individually.

☐ Practice `terraform destroy -target <resource>` to remove just one resource, confirm via `terraform state list` that the other resource survives, then clean up the rest with a plain `terraform destroy`.

☐ Deliberately break the HCL syntax in a `.tf` file and run `terraform validate` — confirm it catches the error before you ever get to `plan`.

☐ Run `terraform fmt` on a deliberately messy/inconsistently-indented `.tf` file and confirm it reformats to canonical HCL style.

What I practiced

- Walked through the full 5-step workflow conceptually (Write → Init → Plan → Apply → Destroy) before touching any real commands.
- Ran `terraform init`, `plan`, `validate`, `fmt`, and `providers` against a real `ec2.tf` config.
- Set up Terraform's shell autocompletion via `.bashrc` + `source ~/.bashrc`.
- Applied a config with `-auto-approve`, then inspected the result with `terraform state list`, `terraform show`, and `terraform state show aws_instance.web`.
- Selectively destroyed resources one at a time with `terraform destroy -target <resource>` instead of tearing down the whole stack at once, confirming each removal via `terraform state list`.

Merged into the master reference notes: terraform-master-notes.md — new "1. Terraform Workflow" section.